

An Overview of the HDF5 Technology Suite and its Applications

Mike Folk, Gerd Heber,
Quincey Koziol, Elena Pourmal,
Dana Robinson
The HDF Group
1800 S. Oak St., Suite 203
Champaign, IL 61820, U.S.A.
+1 (217) 531-6100

[mfolk,gheber,koziol,epourmal,
derobins]@hdfgroup.org

ABSTRACT

In this paper, we give an overview of the HDF5 technology suite and some of its applications. We discuss the HDF5 data model, the HDF5 software architecture and some of its performance enhancing capabilities.

Categories and Subject Descriptors

D.4.2 [Operating Systems]: Storage management - *secondary storage, storage hierarchies*.

D.4.3 [Operating Systems]: File System Management - *access methods, file organization*

H.2.8 [Information Storage and Retrieval]: Database Application – *scientific databases*

H.3.2 [Information Storage and Retrieval]: Information Storage - *file organization*

J.0 [Computer Applications]: General

General Terms

Management, Performance, Design, Reliability, Standardization.

Keywords

HDF5, Databases, Data Models, Data Management.

1. INTRODUCTION

The HDF5 technology suite [1], [2], [3], [4] consists of a data model, a library, and a file format for storing and managing data. It supports an unlimited variety of datatypes, and is designed for flexible and efficient I/O and for high-volume and complex data. HDF5 is portable and is extensible, allowing applications to

evolve in their use of HDF5. The HDF5 technology suite also includes tools and applications for managing, manipulating, viewing, and analyzing data in the HDF5 format.

The paper is organized as follows: In section 2, we describe the HDF5 data model and discuss the particulars of HDF5 array variables. Section 3 contains an overview of the-HDF5 software architecture. In section 4, we discuss certain performance enhancing capabilities like support for parallel I/O and indexing. Some applications using HDF5 are highlighted in section 5 and we touch upon a few challenges on the interface between HDF5 and other technologies in section 6.

2. DATA MODEL

The HDF5 data model does not command the sparkling simplicity of the relational model and that is perceived a blessing by some and a challenge by others. The HDF5 data model defines HDF5 information sets. *An HDF5 information set (info set) is a container for annotated associations of array variables, groups, and types.* In the HDF5 data model the container aspect of an HDF5 info set is represented by an HDF5 file with a designated root. An HDF5 file contains array variables, groups, and types which in the HDF5 data model are known as HDF5 datasets, HDF5 groups, and HDF5 datatype objects, respectively. The HDF5 data model defines simple and extended link mechanisms for creating associations between HDF5 information items. Finally, the HDF5 data model defines a facility to annotate HDF5 information items using HDF5 attributes.

There is substantial information in an HDF5 info set that is not explicitly cast in terms of values of array variables and as such is a violation of Codd's information principle [5]. The rich "superstructure" of an HDF5 info set is a departure from the "flat", relational variables only structure of the relational model.

The remainder of this section is a more detailed discussion of several kinds of HDF5 information items.

2.1 HDF5 Datasets

HDF5 datasets are array variables whose data elements are logically laid out or shaped as a multidimensional array. An HDF5 dataspace captures its *rank* (number of dimensions) and current and maximum extent (number of elements) in the respective dimensions. HDF5 datasets can be thought of as *located content* variables [6]. Their value, the totality of data elements of a certain HDF5 datatype (see section 2.3), is spread

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

AD 2011, March 25, 2011, Uppsala, Sweden.

Copyright 2011 ACM 978-1-4503-0614-0/11/03 ...\$10.00

across the sites of a multidimensional (up to rank 32, in HDF5), rectilinear, integer lattice.

In Figure 1, the value of an HDF5 dataset (array variable) is illustrated as the graph of a function defined on the HDF5 dataspace with values in an HDF5 datatype. HDF5 datasets can be viewed as relational variables with the HDF5 dataspace as a

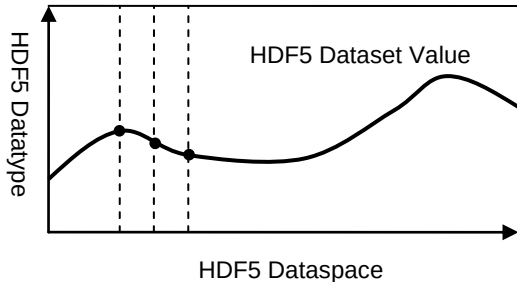


Figure 1. Value of an HDF5 Dataset (variable)

system-defined candidate keyspace.

An HDF5 dataset may grow and shrink within the limits of its maximum extent. Depending on the *storage layout strategy* the maximum extent may be unlimited. Currently, the following options are supported:

- *Contiguous*. Array elements are laid out as a single sequence in the HDF5 array database.
- *Chunked*. Array elements are laid out as a collection of fixed-size regular sub-arrays, or chunks.
- *Compact*. Small HDF5 datasets (less than 64KB total) are laid out in a way that all array elements can be read as part of the array variable's metadata or header (HDF5 datatype, dataspace and other pieces of metadata) retrieval.

Specifying a storage layout strategy opens the door for additional capabilities and potentially higher performance for certain operations. For example, for HDF5 datasets with a contiguous layout strategy, nearly constant access time to any element in the array and zero overhead for locating elements in the dataset is assured. For HDF5 datasets with a chunked layout strategy, HDF5 supports *unlimited* extents for none, some, or all of its dimensions.

HDF5 also supports a filter pipeline (see the 'Filter Pipeline' stage in Figure 4) that provides the capability for standard and customized raw data processing during I/O operations. The HDF5 library is distributed with a small set of standard filters such as compression (gzip, SZIP, and a shuffling algorithm) and error checking (Fletcher32 checksum). For further flexibility, the library allows an application to extend the pipeline through the creation and registration of customized filters which in turn are applied independently to each chunk of an HDF5 dataset. Currently, the HDF5 library does not support filters for contiguous HDF5 datasets because of the difficulty of implementing random access for partial I/O (see section 3.1). HDF5 compact dataset filters are not supported because it would not produce significant results.

Chunked HDF5 datasets offer great flexibility and may substantially improve access to slices of array variables.

Additional performance gains are possible when chunking is used in conjunction with compression since only referenced chunks need to be uncompressed. Chunked HDF5 datasets are not a fire-and-forget solution: chunk sizes must be chosen carefully and there is a tradeoff between faster (chunk-aligned) I/O and added functionality (filters) on the one hand, and the overhead for accessing and maintaining an index to locate the array variable's chunks in the HDF5 file.

Several RDBMS support certain external, non-relational content interfaces, e.g., Informix' Virtual Table Interface. [7] Similarly, an HDF5 dataset may be "backed" by external, non-HDF5 content, which allows for sharing data with applications unaware of HDF5. Such an array variable is referred to as an HDF5 dataset with an *external storage layout*.

In section 0, we give an overview of the operations supported for creation, reading, update, and deletion of HDF5 datasets.

2.2 HDF5 Groups

HDF5 groups are akin to directories in a file system or extended links in XLink [42]. An HDF5 group is an *explicit* representation of an association between zero or more HDF5 information items such as HDF5 datasets, HDF5 groups, and HDF5 datatype objects (see section 2.3). The HDF5 data model does not define or sanction a specific meaning of 'HDF5 group G represents an association between HDF5 information items A, B, C etc.' Possible interpretations include 'G is the parent or context of A, B, C etc.' or 'A, B, C etc. are members of G', but that interpretation is completely up to an application or user. The association or relationship represented by an HDF5 group is made explicit through a collection of *named* links to the participating

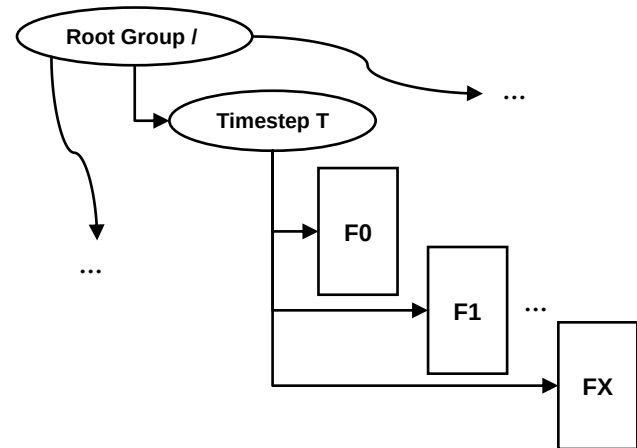


Figure 2. Schematic of Timestep

HDF5 information items.

In each HDF5 file there is exactly one HDF5 group called the *HDF5 root group*. Using or following a link in an HDF5 group is called *traversal*. Link traversal from the HDF5 root group generates the *HDF5 information set graph*, a rooted, directed, connected graph. (Strictly speaking it is a *multi-graph* since multiple edges between the same pair of nodes are allowed.) There exists a directed path from the root to each of its nodes. Note that graphs containing cycles are permitted and that an HDF5 group, HDF5 dataset, or HDF5 datatype object can be linked to more than one HDF5 group.

HDF5 Link Type	Destination via	Ref. Count Change
Hard Link	Address/Identifier	Yes
Soft Link	HDF5 Pathname	No
External Link	File/HDF5 Pathname	No
User-defined Link	User-defined	Maybe

Table 1.

Users and applications may refer to HDF5 information items via the familiar (absolute and relative) HDF5 *pathnames* – concatenated link names interspersed by the link name separation character ‘/’. As an example, consider a simulation of a time-dependent physical process. Under these circumstances it is fairly common to create an HDF5 file which links the HDF5 datasets holding the values of certain discretized physical fields F at a given logical time step T to an HDF group which represents that very time step. This arrangement is shown in Figure 2, where oval shapes symbolize HDF5 groups and rectangles represent HDF5 datasets. [18]

2.3 HDF5 Datatypes

HDF5 offers a type system of nearly unlimited flexibility. As we saw in section 2.1, the (non-scalar) HDF5 array variable type has two key ingredients: an HDF5 dataspace which describes its shape, and an HDF5 datatype which describes the type of its data elements. Ten families or classes of HDF5 datatypes are currently supported: integer, floating-point, string, bitfield, opaque, compound, reference, enum, variable-length sequence and array. For the most part, the type instances of these families are what the family name suggests. With the exception of the compound class (which can be thought of as a tuple types) they all contain scalar types. The reference family contains two types, the HDF5 object reference datatype and the HDF5 region reference datatype (see section 4.2). A variable of type HDF5 object reference datatype holds a reference or pointer to an HDF5 information item. Unlike an HDF5 group, which is an explicit representation of an association between HDF5 information items, an HDF5 dataset or attribute of type HDF5 object reference datatype can be treated by a user or application as an *implicit* representation of such an association. It is *not*, however, an association in the sense of the HDF5 data model.

The type generators of the compound, variable-length sequence, and array families have one or more HDF5 datatype parameters and the HDF5 datatype framework supports arbitrary combinations/nesting depths of HDF5 datatypes, e.g., a field in a compound HDF5 datatype may be of an array HDF5 datatype whose elements in turn are of a variable-length sequence HDF5 datatype and so on.

Besides figuring in the definition of HDF5 datasets and attributes, HDF5 datatype items may be linked to HDF5 groups in which case they are referred to as HDF5 datatype objects or HDF5 named datatypes.

2.4 HDF5 Links

Currently, HDF5 supports unidirectional, single source/single destination links only. Furthermore, the source must be an HDF5 group. HDF5 links can be distinguished by the way they reference their destination and whether they affect the commitment state (reference count) of an HDF5 information item. In Table 1, an overview of HDF5 link types is given.

An HDF5 dataset, group, or datatype object is *committed* to an HDF5 info set by linking it to at least one HDF5 group using an HDF5 hard link. An HDF5 hard link refers to its destination through the HDF5 information item’s object identifier.

HDF5 soft and external links are so-called *symbolic links* because they refer to their destination via an HDF5 pathname or a file name/HDF5 pathname combination. Unlike HDF5 hard links they do not affect the commitment state of their destination. The destination of an HDF5 symbolic link may not exist at link creation time, may change or even cease to exist during the link’s lifetime. It is possible, for example, to create an HDF5 “stub” info set that contains nothing but HDF5 groups, attributes, and external links which acts as a table of contents or lookup.

HDF5 version 1.8.0 saw the introduction of *user-defined links*. HDF5 user-defined links may be symbolic and they may or may not modify their destinations commitment state.

2.5 HDF5 Attributes

The HDF5 data model defines a mechanism called HDF5 attributes for annotating HDF5 datasets, groups, and datatype objects. For logical and implementation reasons, the name of an HDF5 attribute must be unique in the scope of the HDF5 item it is attached to. HDF5 attributes can be thought of as name/value pairs. They are akin to HDF5 datasets in that an HDF5 dataspace and datatype are required for their definition, but they perform a different function – annotation – in an HDF5 info set. An HDF5 attribute cannot have HDF5 attributes itself and is generally intended for small, ancillary data. (Currently, unlimited extents and compression are not supported for HDF5 attributes.) For example, the HDF5 group `/Timestep 10` in Figure 2 might have an HDF5 attribute with the physical time value corresponding to the logical step number 10.

3. HDF5 SOFTWARE ARCHITECTURE

In Figure 3, a schematic of the HDF5 software architecture is shown. Typically, applications, tools, and high-level APIs access HDF5 files through the HDF5 library API. RDBMS users can think of this layer as an ODBC-style interface. The core library is written in C, but there are language bindings available for other languages including FORTRAN, C++, Java, Python, Perl, Ada, and the .NET Framework. Note that not all bindings support the full API provided in the core HDF5 library.

HDF5 files are stored in a *self-describing* format [8] which combines data and metadata. However, it is the “marriage of a format and an access library” that “allows one to access the data without knowing anything about the actual representation of the data or the layout of the file.” [41]

3.1 HDF5 Library Interface

The HDF5 library is the main vehicle for accessing and managing the various kinds of items in an HDF5 file. [3] In this section, we focus on the most important create/read/update/delete operations for HDF5 datasets. For the lack of an elegant array calculus or algebra, we have to resort to rather low-level metaphors.

As outlined in section 2.1, upon creation of an HDF5 dataset the following parameters must be specified:

- the data element type, an HDF5 datatype
- the shape of the underlying multidimensional array, including current and maximum extent, an HDF5 dataspace

- the storage layout strategy (contiguous, chunked, compact)
- filters (e.g., compression, checksum, encryption)
- external layout

Once created, the HDF5 dataset must be committed to the HDF5 array database by linking it to at least one HDF5 group. (Uncommitted variables are abandoned upon closure of the underlying HDF5 file.)

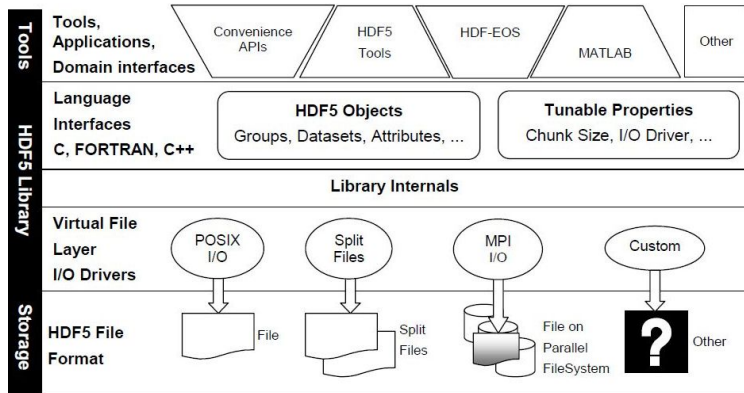


Figure 3.

When reading or updating an HDF5 array variable, the library goes through certain processing steps that we refer to as the *HDF5 data pipeline* (see Figure 4). On either end of the pipeline is an array variable that is either read from or written to. In a typical scenario, one of them is stored in an HDF5 file in a file system (e.g., ext3, NTFS, GPFS) and the other is mapped into an application's address space (a glorified memory buffer). Let V and W denote array variables where, for example, V is stored in a file system and W is in memory. A read or write operation from V into W can be thought of as an update of the form (SQL pseudocode)

```
UPDATE W SET W = T(V)
FROM V JOIN W ON S(V#) = W#
WHERE P(V#) AND Q(W#)
```

Both V and W have associated dataspace denoted by $V\#$ and $W\#$, respectively, and we took the liberty to use the same symbol for the system-defined key (see section 2.1). The HDF5 library supports *partial I/O*, i.e., read or write operations may involve only subsets of data elements of V and W . The selection of these subsets is controlled by predicates P and Q which are defined on $V\#$ and $W\#$. The HDF5 library assumes that the selections are of the same cardinality, i.e., $\# \{P(V\#)\} = \# \{Q(W\#)\}$, and that there is a bijection S between $\{P(V\#)\}$ and $\{Q(W\#)\}$. Finally, there might be a value transformation T , e.g., offsetting or scaling, defined on the components of data elements of V . This transformation may involve type promotions and lower-level representation changes like endianness.

In practice, the predicates supported by the HDF5 library are less general than the pseudocode might suggest. The HDF5 library supports HDF5 selections in the form of HDF5 hyperslabs and HDF5 point sets. HDF5 hyperslabs are the higher-dimensional analogue of a one-dimensional [start, stride, count, block] pattern. The bijection S mentioned above is given by the default traversal orders on HDF5 hyperslabs and point sets.

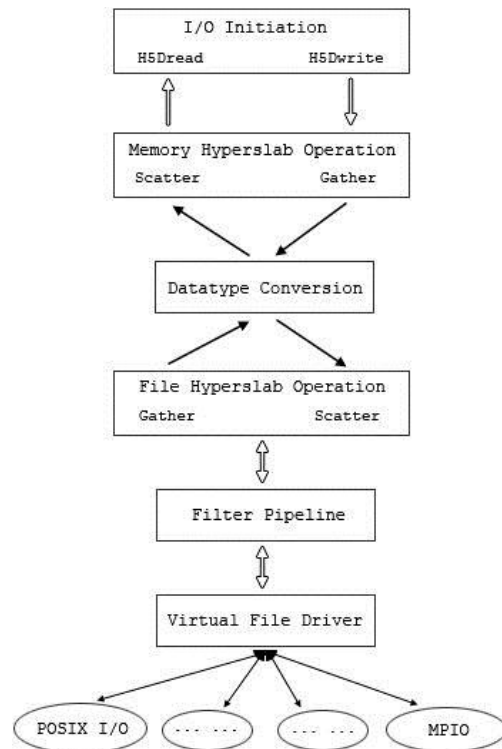


Figure 4. HDF5 Data Pipeline

The SQL pseudocode does not account for the 'Filter Pipeline' and 'Virtual File Driver' (VFD) stages in Figure 4. The filter stage deals with HDF5 filters.

An HDF5 array variable is deleted by removing all links from HDF5 groups. Currently, the freed space can be reclaimed as long as the underlying HDF5 file remains open. (See section 3 of reference [9] for a more detailed discussion.) In a future release, we will implement a more flexible solution.

3.2 Tools

Several command line tools are part of the HDF5 distribution. [11] The tools assist the user in a variety of activities, including examining or managing HDF5 files, converting raw data between HDF5 and other special-purpose formats, moving data and files between the HDF4 and HDF5 formats, measuring HDF5 library performance, and managing HDF5 library and application compilation, installation and configuration. For example, there is a tool called `h5diff` that can be used to compare two HDF5 files and report differences.

Another popular tool is HDFView [12] (see Figure 8 for a screenshot), a GUI tool for browsing and editing HDF5 (and HDF4) files. Using HDFView, you can:

- view a file hierarchy in a tree structure
- create a new file, add or delete groups and datasets
- view and modify the content of a dataset
- add, delete and modify attributes
- replace I/O and GUI components such as table view, image view and metadata view

Other applications, such as MATLAB [13], Mathematica [14], VisIt [15], and EnSight [16], that can access data stored in HDF5 files provide additional capabilities for browsing, modification, and visualization.

3.3 High-Level (HL) Interfaces

HDF5 HL interfaces are akin to the familiar DataBlade [33] or Data Cartridge [34] modules found in RDBMS. The HL interfaces [17] were designed to

- Facilitate and simplify access to array variables in HDF5 files
- Provide standardized representations for storing entities including raster images, palettes, tables, and streaming data.

For example, the HDF5 Packet Table API [37] was designed to provide high-performance, reduced table functionality for storing streaming data in HDF5 (see Figure 5).

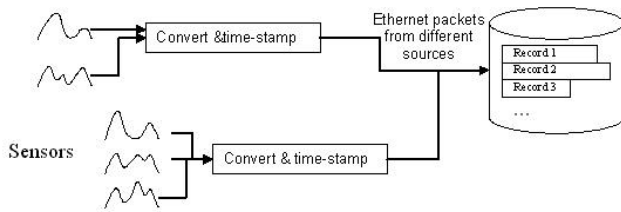


Figure 5.

An HDF5 Packet Table may store fixed size or variable size records. It is represented by a one-dimensional HDF5 array variable with elements of any HDF5 datatype that describes the record (vs. a compound HDF5 datatype for the HDF5 Table API [29]). Certain tradeoffs had to be made to meet the performance goals. The supported operations for HDF5 Packet Tables include table creation, and appending and reading records. There is no support for record deletion or to modify a subset of fields. Unlike HDF5 Tables, there are no special HDF5 attributes attached to an HDF5 Packet Table. HDF5 attributes may be added to an HDF5 Packet Table via the standard, “low-level” HDF5 library API.

4. PERFORMANCE

The HDF5 library lets users exploit parallelism at the CPU and file system levels to achieve better I/O performance. Users have also been successful implementing more or less generally applicable schemes for indexing HDF5 datasets.

The HDF5 library uses a single global semaphore to protect internal data structures from being modified by multiple threads of execution. Although this provides thread-safety, it does not allow concurrent access to HDF5 files. Future work will move this locking mechanism onto individual data structures within the library, allowing concurrent access by multiple threads.

4.1 Parallel File I/O

In MPI-based [35] parallel applications the HDF5 library can be used for accessing HDF5 files in parallel. The primary goal is to

enable fast, parallel I/O to large HDF5 datasets through a standard parallel I/O interface, MPI-IO, and taking advantage of parallel file systems such as GPFS and Lustre [19,35].

Parallel access can be independent or collective. In *independent* mode, individual MPI processes perform I/O operations independent of and unaffected by other MPI processes in the same application accessing the same HDF5 file. In *collective* mode, parallel access is a highly coordinated, cooperative effort involving all MPI processes in an application.

The two modes differ in programming complexity, have different performance implications, and are constrained in different ways by the current parallel HDF5 implementation (but not the HDF5 data model or format!). For example, it is currently necessary to create, modify, or delete objects in an HDF5 file in collective mode. This way a globally consistent view of an HDF5 file’s content can be ensured. A common strategy in independent mode is to pre-create the overall structure of the HDF5 file and then perform read and/or write operations as needed without any further synchronization or coordination between the MPI processes. The burden of avoiding race conditions is on the programmer. Because of a lack of an explicit I/O pattern, dynamic optimizations on the target file system are limited and, in the worst case, may exacerbate resource contention. When performing I/O collectively there is a global perspective and hence a better chance for leveraging a parallel file system’s buffering and caching mechanism. On the other hand, the added communication and synchronization overhead may reduce performance and scalability.

4.2 Indexing

In this section, we focus on indexing individual array variables. Currently, the HDF5 library does not offer a standardized API for indexing HDF5 datasets. This is not due to a lack of interest in the HDF5 users community. In fact, there have been several efforts (see for example [18], [22], [23]) to create more or less generally applicable solutions.

The highly developed indexing support found in enterprise RDBMS offers some point of reference for comparing the indexing requirements between relational variables and HDF5 datasets. In section 2.1, we mentioned that HDF5 array variables can be viewed as relational variables with a system-defined candidate key provided by the courtesy of the underlying HDF5 dataspace. In view of the rich arithmo-geometric structure of the HDF5 dataspace, this is somewhat of an understatement. Viewing an HDF5 dataset as a relational variable, there are two subsets of relational attributes, a subset A of HDF5 dataspace-aware attributes (e.g., integer lattice coordinates) and its complement B. Simple queries targeting attributes in A can take advantage of the quasi-random access or HDF5 region references (see below), and traditional indexing techniques may work well for queries targeting attributes in B. For a combined query targeting A and B simultaneously there should be a class of HDF5 dataspace-aware indexes (self-organizing maps, wavelets?), which also performs well in higher-dimensional settings (rank > 3).

In relational databases, indexes are defined through user-visible components. The main reason is *Codd's Information Principle* [5]: "All information in the database at any given time must be cast explicitly in terms of values in relations and in no other way." This creates a certain tension between types and relational variables (see Appendix B "The Database Design Dilemma" in [5]) and determines what can be indexed. The tremendous flexibility offered by scalar HDF5 datatypes is a slippery slope that may tempt users to create datasets that are mere "collections of encapsulated monstrosities" and as such are *unindexable* for querying purposes. The following suggestion from [5] applies to HDF5 infosets the same way it does to relational database design: "Overall, we suspect that the most appropriate design choices will emerge if careful consideration is given to the distinction between (a) declarative sentences in human language and (b) the vocabulary used in the construction of such sentences... Domains

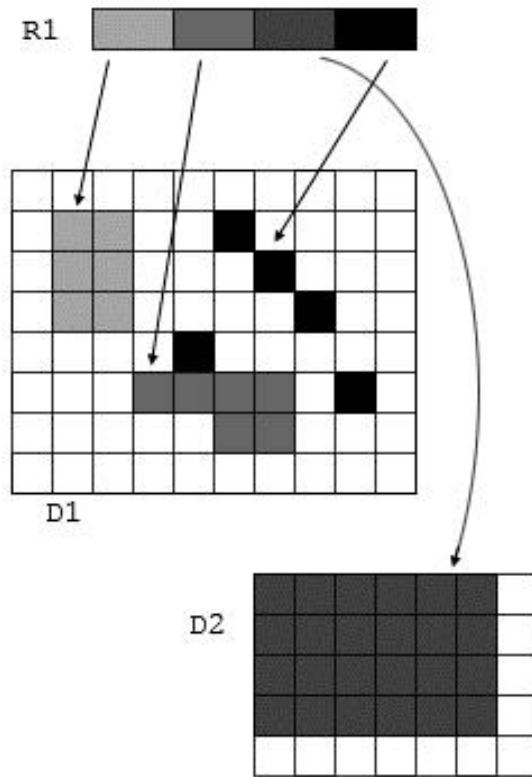


Figure 6. HDF5 Region References

(types) give us values that represent things we might wish to refer to; relations give us ways of referring to those things in utterances about them."

Probably most users encounter HDF5 files in read-mostly environments with complex (i.e., involving mathematical expressions of the column values) ad hoc queries. [24] Unlike tables in transaction processing systems, HDF5 datasets and their associated indexes typically do not change after their creation. This opens the door for indexing techniques that otherwise would be costly to maintain under deletion, insertion, and/or update.

The high-level processing structure of an index-assisted HDF5 dataset read operation is very similar to executing a `SELECT` in an RDBMS. The query is expressed as a truth-valued expression in terms of user-visible components such as $(energy > 10^5)$

AND $(70 < pressure < 90)$. An *optimizer* (programmer or system component) decides if there is an index (one or more) such that an efficient data access path can be devised. Scanning certain *thin slices* of the index results in an HDF5 selection (hyperslab, point) which is in turn used in the actual HDF5 dataset read operation.

HDF5 offers a convenient mechanism, HDF5 *region references*, for persisting HDF5 dataspace selections, see Figure 6. D1 and D2 are two-dimensional HDF5 datasets and R1 is a one-dimensional HDF5 dataset or attribute containing four HDF5 region references to HDF5 dataspace selections in D1 and D2. HDF5 region references can be de-referenced and be used to read the selected data elements.

Unsurprisingly, there is no one-size-fits-all indexing solution for HDF5 datasets that is (1) effective for low- and high-dimensional data and all kinds of queries, (2) can be created with a reasonable amount of resources in good time, and (3) is of a justifiable size. (As stated earlier, updates are not a concern at this point.) In reference [22], an indexing library based on R*-trees for HDF-EOS (see section 5.2) data stored in HDF4 and HDF5 formats is described. In reference [18], a query acceleration technique for HDF5 datasets based on bitmap indexes is presented. The authors demonstrate the effectiveness of bitmap indexes for complex ad hoc queries on large HDF5 datasets, confirming similar results from the RDBMS community [25]. The authors of [23] reach a similar conclusion regarding the suitability of their optimized partially sorted indexes (OPSI) which are based on a radically different approach. If OPSI presents a first example of HDF5 dataset indexes not commonly found in RDBMS, nested containment list (NCList) based indexes [26] are another. (A variation of NCList indexes has been implemented for the BioHDF project [27].)

"Our ultimate goal is to automate the process of generating indexing structures for various self-describing scientific datasets, using meta-information that can be automatically extracted from the metadata within the datasets, augmented with semantic information provided by developers or users of the scientific data file formats." [22] From a user's perspective, this is the ideal state of indexing affairs. For the maintainers of HDF5 the challenge of a "least common denominator problem" remains: the challenge of identifying domain independent mechanisms and facilities that deliver maximum benefits to most HDF5 users.

5. APPLICATIONS

Application and library developers choose HDF5 for a variety of reasons which may include one or more of the following:

- ☐ The simplicity and pragmatism of its data model and/or its proximity to their own data model(s)
- ☐ The longevity of a stable, open, self-describing, platform-independent, binary format
- ☐ The performance enhancements for sequential and parallel computing
- ☐ The gains in interoperability with applications already using HDF5
- ☐ The gains in productivity by standardizing and sharing non-problem oriented code and by eliminating redundant development

- The availability of commercial and freely available visualization and high-performance data analysis tools including MATLAB, Mathematica, IDL, EnSight, ParaView, and Visit
- The advantages of a de facto standard adopted by many organizations and science and engineering communities
- Available support through The HDF Group's Help Desk, mailing lists, and users forum
- Support for high-level languages such as Python and R

Like a few other technologies, HDF5 has shown its potential for transforming research communities or small industries by instigating or changing data management practice. Below we will look at two applications of HDF-EOS/netCDF4 and BioHDF. HDF-EOS is a well-established format and widely used in the earth science community. BioHDF is a nascent format intended to address the bioinformatics data deluge problem. Before discussing the applications we briefly describe an analysis technique that is common to many applications of HDF5.

5.1 HDF5 Profiles

Using HDF5 can be compared to a translation project. It is about mapping application concepts onto the HDF5 data model. This process is necessarily fraught with ambiguity and that, if at all, can be automated only for rather simple scenarios. Trying to answer a few questions may help guide the translation between different conceptual frameworks [44]:

1. What are the *basic facts* that we are trying to represent?
2. What *constraints* must be imposed to exclude non-grammatical and nonsensical statements?
3. How does the translation facilitate the *derivation* and discovery of new facts?

Applications using HDF5 on a large scale tend to codify and materialize their domain specific mapping or translation in the form of high-level libraries and tools. Accompanying documentation will list structural (HDF5 groups, links) requirements, "magic" HDF5 attributes (see section 3.3), HDF5 datatype and link naming conventions, as well as other indicators that enable an application to recognize the structure of an HDF5 file as compliant with the domain standard. Taken together these assumptions and stipulations constitute a so-called *HDF5 profile*. A simple example of such a profile is the HDF5 Image and Palette specification. [28] Unsurprisingly, considering the domain diversity and the vagueness of the definition, there's no specification or standard governing the creation and compliance check for HDF5 profiles.

XML schema validation is probably the first and most mature candidate that comes to mind when looking for a model to emulate. The vocabulary and grammar offered by XML schema is typically insufficient to express both the structure of the basic facts and the structural constraints (see [20] for a discussion). Of course, we would like to employ the highly-developed XML machinery for this and the situation can be remedied by leveraging XQuery [21] to formalize what we refer to in [20] as *HDF5 constraints*. The validity checks for HDF5 constraints are highly mechanized and can be fully automated. In Figure 7, the high-level structure of this HDF5 constraint (satisfaction) validation pipeline is shown. The domain knowledge is wrapped into a module of tests expressed as Boolean XQuery functions (*Constraints.xquery*). Reference [20] does not define what

an HDF5 profile is. It proposes a framework to define and validate HDF5 constraints which can be used to accomplish most of the tasks that are at the heart of the somewhat elusive HDF5 profile.

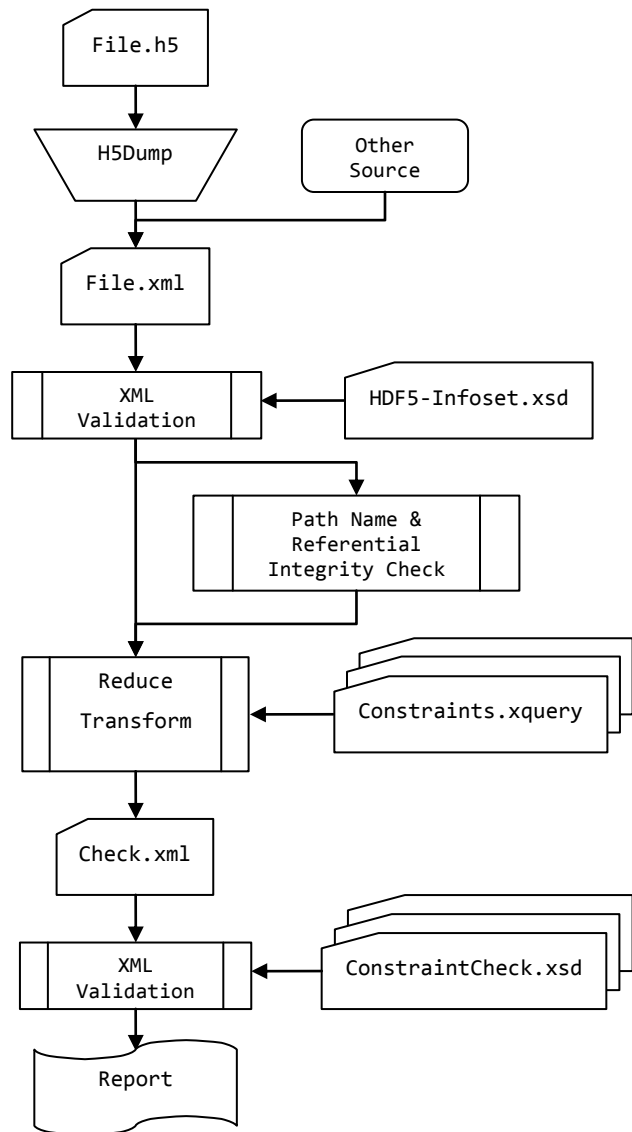


Figure 7. HDF5 Constraint Validation Pipeline

5.2 HDF-EOS5 and netCDF-4

For almost two decades, NASA's Earth Observing System (EOS) [36] has been using HDF (HDF4 and HDF5) for archiving and distributing remote sensing data. A constellation of EOS satellites monitors the world's biosphere, atmosphere, land surface and oceans, gathering environmental data that is critically important to current and future research on global climate change. By the end of the EOS program in 2015, NASA's archives will contain more than 15 petabytes of data in the HDF format. Between 1 October 2008 and 30 September 2009 there were more than 910,000 distinct users of EOSDIS data and services with an average daily distribution volume to end users of 6.7 TB [51].

To achieve a better interoperability between earth science applications and to facilitate access to data stored in HDF, the

NASA earth sciences community created a data model and I/O library called HDF-EOS [36]. The HDF-EOS data model is designed to support earth science data structures such as Swath, Grid, and Point; the HDF-EOS API provides a standard way for storing such structures in HDF. The latest EOS mission “Aura” chose the version of HDF-EOS based on HDF5 as a storage format for the environmental data it gathers.

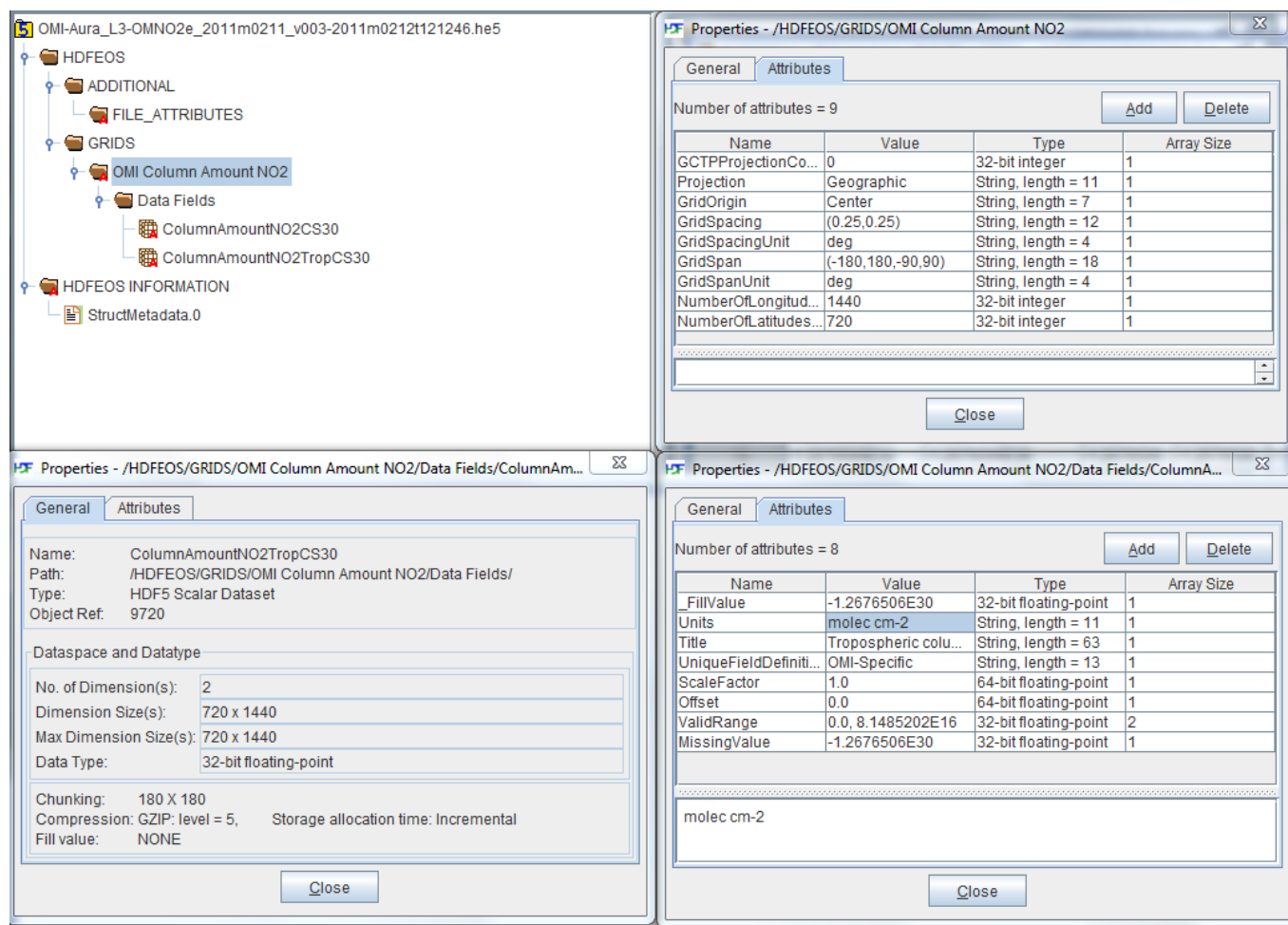
The successful deployment of HDF technologies by the EOS Data and Information System (ESDIS) promoted further usage of HDF5 and prompted NASA to recommend HDF5 and HDF-EOS5 as standard formats for storing earth science data. The successor to the EOS program, the Joint Polar Satellite System (JPSS), has chosen HDF5 as the storage format for its data products that will be delivered to its customers in real-time. As

weather events, thus reducing the potential loss of human life and property and advancing the national economy.”

For many years, climate and weather prediction data models used the netCDF library and file format that were especially well suited for representing and storing gridded outputs [39].

The netCDF data model is simple yet powerful [46]. A netCDF file may contain one or more array variables, dimensions, and attributes; variables may share dimensions, indicating a common grid. Like HDF5, netCDF is also very popular for archiving many kinds of observational data in Earth Science.

New requirements for running climate and weather prediction models in HPC environments triggered the work on a new version of netCDF, netCDF-4, that uses HDF5 as the underlying file



stated on the NOAA website [38] “...JPSS will continue to address NOAA’s requirements to provide global environmental data used in numerical weather prediction models for forecasts, as well as provide space weather observations, search and rescue detection capabilities, and direct read-out and data collection products and services to customers. Data and imagery obtained from the JPSS will increase timeliness, accuracy, and cost-effectiveness of public warnings and forecasts of climate and

format and I/O library [45]. NetCDF-4 combines an enhanced netCDF data model with the high-performance capabilities of HDF5, including HDF5 groups, the rich set of HDF5 datatypes, data compression, and parallel I/O. It also maintains compatibility with existing netCDF files and applications. NetCDF-4, as never before, enhances the usability of earth sciences observational data in climate and weather prediction models run in HPC environments.

5.3 BioHDF

Bioinformatics data storage suffers from a large number of flat, text-based legacy file formats. Although these file formats may be adequate when relatively small amounts of data must be stored or queried, they do not scale well and have proven inadequate for storing the increasing quantities of data that can be emitted by recent instruments. The most striking example of this has been the data management crisis that the "next-generation" DNA sequencing (NGS) community is experiencing [63], [60]. In less than a decade, DNA sequencers have gone from first-generation instruments that could produce a few tens of megabytes of raw data per day to second-generation sequencing instruments which can produce tens of gigabytes per day with even higher throughput third-generation instruments on the horizon. This deluge of data has overwhelmed the existing text-based data analysis pipelines.

Some early solutions to this problem have appeared. The text-based sequence-alignment map (SAM) format and its binary counterpart (BAM) [58] are popular for storing both the sequencer output (reads) and a mapping of these reads to a reference DNA sequence (alignments), the TABIX format [57] uses some of the underlying architecture of BAM to store arbitrary tab-delimited files and BigWig [56] stores reference-ordered numerical data tracks. These newer formats can be stored in a more efficient binary format, support compression and, most importantly, include indexes for fast data access. Although an improvement over their text-based predecessors, these file formats are just that – flat, inflexible data formats which can be difficult to extend and adapt to new requirements. What is needed is a flexible, high-performance data storage system which can scale to NGS-levels of data.

BioHDF [59] addresses these deficiencies by providing an HDF5-based data storage system for the bioinformatics community with an initial focus on NGS data storage. BioHDF includes a data model, which maps NGS data objects, such as reads and alignments, to HDF5 data objects; an API written in the C programming language and command-line tools which allow data import and export. The data model upon which BioHDF is based is designed to be flexible and extensible, a job which is made easier given the self-describing nature of data stored in HDF5. A given NGS data type (e.g. alignments) is stored using a predefined scheme via its own API. The flexible layout allows data in the file to be organized in a way that meets the needs of a particular experiment. As an example, NGS data for a pharmaceutical study may require data to be segregated by patient, whereas a study that investigates community bacteria in hot springs may require a different layout. A flexible format also allows for future expansion without breaking existing software, which has proven to be a problem for flat files in the past.

6. BEYOND HDF5

HDF-EOS and BioHDF are two examples of large and/or growing, mostly read-only collections of HDF5 files. There are other examples in materials science, the exploration of mineral resources, the financial industry, and performance testing. (More than 200 distinct applications of HDF5 have been identified.) Such repositories pose a substantial management challenge, but, more importantly, a justification challenge, a challenge to demonstrate, in economic and/or intellectual terms, "data productivity" – the derivation of new information and the

discovery of new facts. This challenge puts HDF5's flexibility to integrate with search, query, and other exploratory interfaces to the test.

In this section, we describe two interfaces that facilitate the exploration and exploitation of HDF5 repositories. The first example, BCS UFI [10], is a technology which allows clients to transparently access HDF5 files through a relational frontend. The second example, OPeNDAP [52], [53], is a protocol which, among other things, enables clients to transparently access HDF5 repositories over the network.

6.1 HDF5 as a DBMS Back-End

A DBMS provides a standard query language and a wealth of front-end clients. Arranging for HDF5 files to appear as collections of tables allows custom HDF5 applications to be assembled with minimal or no programming, and a user can then issue queries that combine data stored in HDF5 files with data already stored in conventional database tables.

Barrodale Computing Services (BCS) [47] has developed a plug-in called the Universal File Interface (UFI) [10] that allows the Informix DBMS – through its Virtual Table Interface – to transparently access HDF5 files as if they were conventional tables in an RDBMS; BCS is currently extending UFI to work with most other DBMSs (Oracle, DB2, SQL-Server,...). The UFI architecture is shown in Figure 9.

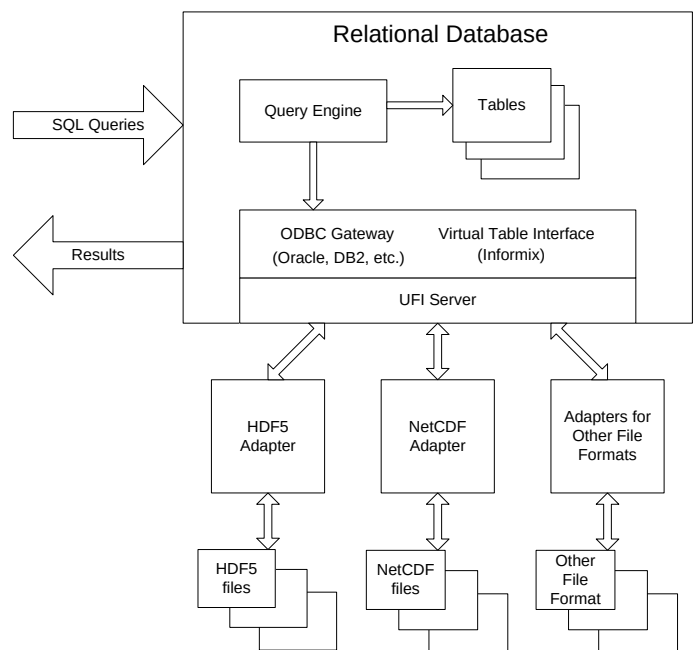


Figure 9. The Architecture of UFI

Using UFI with HDF5 files is a two-step process:

1. Define a virtual table (table name, column names, and column types) and, for each column, define an association between the column and an HDF5 file object (group, dataset, attribute, etc.). This association is defined using an HDF5 pathname, possibly including wildcards.

2. Associate the table with one or more HDF5 files.

Now consider the HDF5 file illustrated in the HDFView screen capture shown in Figure 8. The file described by Figure 8 is a Level 3 (gridded) data product produced from one day's worth of NO₂ measurements [48] made by the Ozone Monitoring Instrument (OMI) [49] aboard the NASA EOS-Aura satellite [50]. The objects shown with folder icons in the directory tree on the top left are HDF5 groups. The two children of the "Data Fields" group are HDF5 datasets. Some groups and datasets have attributes (the ones annotated with tiny red 'A's on lines 4,6,8,9, and 10 in the top left pane of Figure 8); some do not. A description of the "OMI Column Amount NO2" group is shown on the top right, while one of its datasets, and that dataset's attributes are described by the bottom left and right panes, respectively. As is apparent from the attributes shown in the top right pane, the file contains data for the whole world, at a resolution of .25 degrees. Not shown in Figure 8 are the attributes for the "FILE_ATTRIBUTES" group; those attributes include the day/month/year and the satellite orbit numbers of the passes when the underlying measurements were taken. One possible use of the EOS-Aura NO₂ data is to explore how air quality, represented by NO₂ measurements in the HDF5 files, is affected by certain events (such as industrial mills starting and stopping operation) at

identically structured HDF5 files located in some particular directory or directories on the database server.

Suppose that our (real) location and event database tables are defined as follows:

```
TABLE mill_sites(
    mill_name    varchar(100),
    mill_area    polygon)
```

```
TABLE events(
    event_name    varchar(100),
    event_date    date)
```

Then we could issue the following query concerning mills and pollution:

```
SELECT avg(concentration)
FROM no2_concentrations, mill _sites, events
WHERE to_date(year,month,day)
      BETWEEN event_date AND event_date + 10
      AND event_name = 'Mill X Opening'
      AND mill_name = 'X'
      AND contains(mill_area,
                  to_point(latitude,longitude));
```

Column	HDF5 Pathname
lat_index	'/HDFEOS/GRIDS/OMI Column Amount NO2/Data Fields/ColumnAmountNO2TropCS30[link1:\$,link2:*]'
lng_index	'/HDFEOS/GRIDS/OMI Column Amount NO2/Data Fields/ColumnAmountNO2TropCS30[link1:*,link2:\$]'
Latitude	-90.0 + (lat_index * .25)
Longitude	-180.0 + (lng_index * .25)
concentration	'/HDFEOS/GRIDS/OMI Column Amount NO2/Data Fields/ColumnAmountNO2TropCS30[link1:*,link2:*]'
null_value	'/HDFEOS/GRIDS/OMI Column Amount NO2/Data Fields/ColumnAmountNO2TropCS30/ATTR:MissingValue[*]'
Day	'/HDFEOS/ADDITIONAL/FILE_ATTRIBUTES/ATTR:GranuleDay[*]'
Month	'/HDFEOS/ADDITIONAL/FILE_ATTRIBUTES/ATTR:GranuleMonth[*]'
Year	'/HDFEOS/ADDITIONAL/FILE_ATTRIBUTES/ATTR:GranuleYear[*]'

Figure 10.

various locations. The times of these events and their locations might be stored in regular database tables. To do this analysis we would create a virtual table

```
TABLE no2_concentrations(
    lat_index    int8,
    lng_index    int8,
    latitude     float,
    longitude    float,
    concentration double precision,
    null_value   double precision,
    day          char(2),
    month        char(2),
    year         char(4))
```

and use UFI to define mappings between these columns and the HDF5 groups, datasets, and attributes, as shown above in Figure 10, and then tell UFI to base this table on a repository of

This query can be answered very quickly, since UFI can use the *latitude/longitude* - to - *lat_index/lng_index* mappings shown above in Figure 10 to directly access the required elements of the *ColumnAmountNO2TropCS30* dataset. SQL queries such as this are very easy to write and efficient to execute, making UFI an effective tool for exploring large repositories of HDF files, such as those produced by the NASA EOS program.

6.2 HDF-OPeNDAP

For clients accessing remote repositories over the internet it is important to have the ability to locate and retrieve subsets of data of interest rather than being forced to download excessive amounts of unnecessary data. "The Data Access Protocol (DAP) [55] is a framework that simplifies the retrieval of scientific data by allowing users to browse and select only the subset of data they are interested in." [54] OPeNDAP [53] is a client-server architecture which uses DAP for communication. Originally developed for the National Virtual Ocean Data System (NVO DS),

DAP has been adopted in large parts of the climate, oceanography, and meteorology communities. At the heart of the

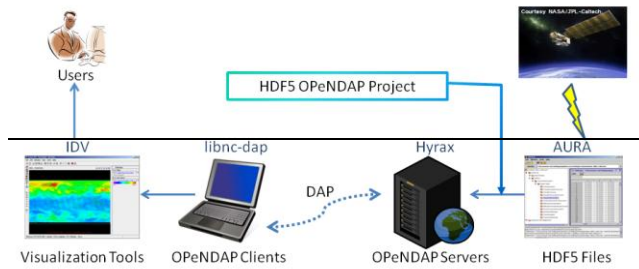


Figure 11. Data Access via OPeNDAP

DAP data model is the concept of a DAP data source, a collection of typed, annotated, and associated variables. DAP defines facilities (request types) to discover and interrogate DAP data sources. HDF-OPeNDAP [52] (see Figure 11) implements an OPeNDAP server for HDF4 and HDF5 files featuring [54]:

- A mapping of atomic HDF5 datatypes to simple DAP data types
- A translation of HDF5 datasets into multidimensional arrays of DAP data types
- A translation of HDF5 attributes into DAP attributes
- A mapping of HDF5 compound datatypes into DAP structures
- A mapping of HDF5 groups to DAP attributes
- A mapping of HDF5 reference datatypes into DAP URLs
- A mapping of EOS grids to DAP grids

As a result, remote access to HDF data at the sub-file level can be gained using popular OPeNDAP clients such as Panoply, IDV, Ferret, and GrADS [61].

7. CONCLUSION

In this paper, we have given an overview of the HDF5 technology suite and some of its applications. HDF5 addresses the following major data challenges:

1. It gives application developers the ability to organize complex collections of data.
2. It provides the means for efficient and scalable data storage and access.
3. It satisfies a growing need to integrate a wide variety of types of data and data sources.
4. It takes advantage of evolving data technologies but shields users and applications.
5. It guarantees the long term preservation of data.

There are plenty of data challenges which go beyond core HDF5 competencies. However, for an important subset of problems dealing with data delivery, search, and query, HDF5 – in conjunction with other technologies ([10], [40], [53], [62]) – is capable of delivering highly competitive solutions.

8. REFERENCES

- [1] <http://www.hdfgroup.org/HDF5/doc/index.html>
- [2] HDF5 User's Guide <http://www.hdfgroup.org/HDF5/doc/UG/index.html>.
- [3] HDF5 Reference Manual http://www.hdfgroup.org/HDF5/doc/RM/RM_H5Front.html.
- [4] Koziol, Q. 2011. *HDF5 Encyclopedia of Parallel Computing*. To appear.
- [5] Date, C. J. and Darwen, H. 1998. *Foundation for Object/Relational Databases – The Third Manifesto*. Addison Wesley.
- [6] Thomsen, E. 2002. *OLAP Solutions: Building Multidimensional Information Systems*. Second Edition. Wiley.
- [7] Brown, P. 2001. *Object-Relational Database Development*. Informix Press.
- [8] HDF5 File Format Specification Version 2.0 <http://www.hdfgroup.org/HDF5/doc/H5.format.html>
- [9] Performance Analysis and Issues <http://www.hdfgroup.org/HDF5/doc/H5.user/Performance.html>
- [10] Universal File Interface (UFI) <http://www.barrodale.com/bcs/universal-file-interface-ufi>
- [11] HDF5 Tools <http://www.hdfgroup.org/HDF5/doc/RM/Tools.html>
- [12] HDFView <http://www.hdfgroup.org/hdf-java-html/hdfview/>
- [13] MathWorks <http://www.mathworks.com/>
- [14] Mathematica <http://www.wolfram.com/>
- [15] VisIt <https://wci.llnl.gov/codes/visit/>
- [16] EnSight <http://www.ensight.com/>
- [17] HDF5 High-level APIs <http://www.hdfgroup.org/HDF5/doc/HL/>
- [18] Gosink, L. et al. 2005. *HDF5-FastQuery: Accelerating Complex Queries on HDF5 Datasets using Fast Bitmap Indices*. <http://crd.lbl.gov/~kewu/ps/LBNL-59602.pdf>
- [19] Mainzer, J. and Koziol, Q. 2010. *RFC: High-Level HDF5 API routines for HPC Applications*. http://www.hdfgroup.uiuc.edu/RFC/HDF5/HPC-High-Level-API/H5HPC_RFC-2010-09-28.pdf
- [20] Folk, M. and Heber, G. and Koziol, Q. 2011. *HDF5 Information Set*. To appear.
- [21] W3C. 2010. *XQuery 3.0: An XML Query Language*. <http://www.w3.org/TR/xquery-30/>
- [22] Nam, B. and Sussman, A. 2003. *Improving Access to Multi-dimensional Self-describing Scientific Datasets*. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.78.2998&rep=rep1&type=pdf>
- [23] Altet, F. and Vilata, I. 2007. *OPSI: The indexing system of PyTables 2 Professional Edition*. <http://www.pytables.org/docs/OPSI-indexes.pdf>
- [24] Chan, C-Y. and Ioannidis, Y. E. 1998. *Bitmap Index Design and Evaluation*. SIGMOD.

- <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.87.1270&rep=rep1&type=pdf>
- [25] Lahdenmaeki, T. and Leach, M. 2004. *Relational Database Index Design and the Optimizers*. John Wiley & Sons, Inc.
 - [26] Alekseyenko, A. V. and Lee, C. J. 2007. *Nested Containment List (NCList): a new algorithm for accelerating interval query of genome alignment and interval databases*. Bioinformatics Vol. 23, No 11, pp. 1386-1393.
 - [27] BioHDF <http://www.biohdf.org/>
 - [28] HDF5 Image and Palette Specification. 2007. http://www.hdfgroup.org/HDF5/doc/HL/RM_H5IM.html
 - [29] HDF5 Table Specification. 2002. http://www.hdfgroup.org/HDF5/doc/HL/H5TB_Spec.html
 - [30] HDF5 Dimension Scale Specification and Design Notes. 2005. http://www.hdfgroup.org/HDF5/doc/HL/H5DS_Spec.pdf
 - [31] Howison, M. et al. 2010. *Tuning HDF5 for Lustre File Systems*. https://secure.nersc.gov/projects/presentations/HDF5_DonofrioNERSC.pdf
 - [32] Introduction to the HDF5 Packet Table API. 2005. http://www.hdfgroup.org/HDF5/doc/HL/H5PT_Intro.html
 - [33] Informix DataBlades. 2010. <http://www-01.ibm.com/software/data/informix/blades/>
 - [34] Data Cartridge – Oracle Wiki. 2010. <http://wiki.oracle.com/page/Data+Cartridge>
 - [35] Message Passing Interface. 2010. <http://www.mcs.anl.gov/research/projects/mpi/>
 - [36] NASA's Earth Observing System . 2010. <http://eospsso.gsfc.nasa.gov/>
 - [37] The ECS SDP Toolkit Home Page. 2010. <http://newsroom.gsfc.nasa.gov/sdptoolkit/TKDocuments.html>
 - [38] NESDIS Satellite Information . 2010. <http://www.nesdis.noaa.gov/SatInformation.html>
 - [39] NetCDF (network Common Data Form). 2010. <http://www.unidata.ucar.edu/software/netcdf/>
 - [40] SciDB. *Overview of SciDB, Large Scale Array Storage, Processing and Analysis*. SIGMOD'10. <http://www.scidb.org/download/sigmod691-brown.pdf>
 - [41] Kuehn, J.A. 1996. *Faster Libraries for Creating Network-Portable Self-Describing Datasets*. Cray User Group
 - [42] XML Linking Language (XLink) Version 1.1. 2010. <http://www.w3.org/TR/xlink11/>
 - [43] *Standardizing the Next Generation of Bioinformatics Software Development with BioHDF*. BioHDF BoF SC09. 2009.
 - [44] Halpin, T. 2001. *Information Modeling and Relational Databases*. Morgan Kaufmann Publishers.
 - [45] *The NetCDF-4 Data Model*. 2010.
 - [46] *The "Classic" NetCDF Data Model*. 2010.
 - [47] Barrodale Computing Services. 2010. <http://www.barrodale.com/>
 - [48] Aura OMI NO2 Level 3 Global (0.25 deg Grids) Data Product-OMNO2e Version 003. http://disc.sci.gsfc.nasa.gov/Aura/data-holdings/OMI/omno2e_v003.shtml
 - [49] NASA. OMI Data Products and Data Access. 2010. <http://disc.sci.gsfc.nasa.gov/Aura/overview/data-holdings/OMI/index.shtml>
 - [50] NASA. Aura. 2010. <http://disc.sci.gsfc.nasa.gov/Aura>
 - [51] Ramapriyan, H.K and Moses, J. 2011. *NASA's Earth Science Data Systems – Lessons Learned and Future Directions*. To appear.
 - [52] HDF OPeNDAP Project. 2010. <http://www.hdfgroup.org/projects/opensdap/>
 - [53] OPeNDAP: Open-source Project for a Network Data Access Protocol. 2010. <http://www.opensdap.org/>
 - [54] Yang, M. and Lee H. 2009. *Using a Friendly OPeNDAP Client Library to access HDF5 data*. The 89th AMS annual meeting.
 - [55] Gallagher, J. et al. 2007. *The Data Access Protocol – DAP 2.0*. ESE-RFC-004.1.1
 - [56] Kent, W.J., Zweig, A.S., Barber, G., Hinrichs, A.S. and Karolchik, D. 2010. *BigWig and BigBed: enabling browsing of large distributed datasets*. Bioinformatics, **26**, 2204-2207.
 - [57] Li, H. 2011. *Tabix: fast retrieval of sequence features from generic TAB-delimited files*. Bioinformatics, **27**, 718-719.
 - [58] Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G. and Durbin, R. 2009. *The Sequence Alignment/Map format and SAMtools*. Bioinformatics, **25**, 2078-2079.
 - [59] Mason, C.E., Zumbo, P., Sanders, S., Folk, M., Robinson, D., Aydt, R., Gollery, M., Welsh, M., Olson, N.E. and Smith, T.M. 2010. *Standardizing the next generation of bioinformatics software development with BioHDF (HDF5)*. Adv Exp Med Biol, **680**, 693-700.
 - [60] Shendure, J. and Ji, H. (2008) *Next-generation DNA sequencing*. Nat Biotechnol, **26**, 1135-1145.
 - [61] Making Science Data Easier to Use with OPeNDAP. 2010. http://wiki.esipfed.org/index.php/Making_Science_Data_Easier_to_Use_with_OPeNDAP#EOS_HDF_Data_Readability_.28without_OPeNDAP.29
 - [62] Open Data Protocol (OData). 2010. <http://www.odata.org/>
 - [63] Kahn, S.D. 2011. *On the future of genomic data*, Science, **331**, 728-729.